

Informe Proyecto 1
Ciencia de datos

Asignatura:

Introducción a la ciencia de los datos

Programa:

Ingeniería de Sistemas

Estudiantes:

Juan Esteban López Aránzazu - 202313026
Daniel Meléndez Ramirez - 202313024
Joan Sebastián Saavedra Perafán - 202313025

Semestre:

Séptimo Semestre

Docente:

Héctor Fabio Ocampo Arbeláez

Universidad del Valle – Sede Tuluá

Informe del proyecto

I. Estructura del proyecto

El proyecto tiene la siguiente estructura de carpetas/archivos

1. config: directorio para la configuración de las variables de entorno
2. data: directorio para guardar los archivos csv
3. db: directorio para la conexión a la base de datos con PostgreSQL
4. graphs: directorio para guardar las gráficas generadas con matplotlib
5. models: directorio con las tablas de la base de datos
6. notebooks: directorio con los archivos jupyter notebook para analizar los datos del archivo csv
7. sql: directorio con los archivos sql para crear las tablas y consultas
8. utils: directorio para los archivos de utilidad para cargar los datos a la base de datos y generar las gráficas en matplotlib

Para ejecutar el proyecto se deben hacer los siguientes pasos:

1. Crear un archivo .env en el directorio raíz, siguiendo la estructura del archivo .env.example
2. Crear un entorno virtual dependiendo del sistema operativo
3. Instalar las dependencias del archivo requirements.txt
4. Ejecutar el archivo main.py

Variables de entorno para la conexión a la base de datos

1. DB_USER: variable de entorno para el nombre de usuario
2. DB_PASSWORD: variable de entorno para la contraseña
3. DB_HOST: variable de entorno para el servidor
4. DB_NAME: variable de entorno para el nombre de la base de datos
5. DB_PORT: variable de entorno para el puerto, el puerto por defecto de PostgreSQL es 5432

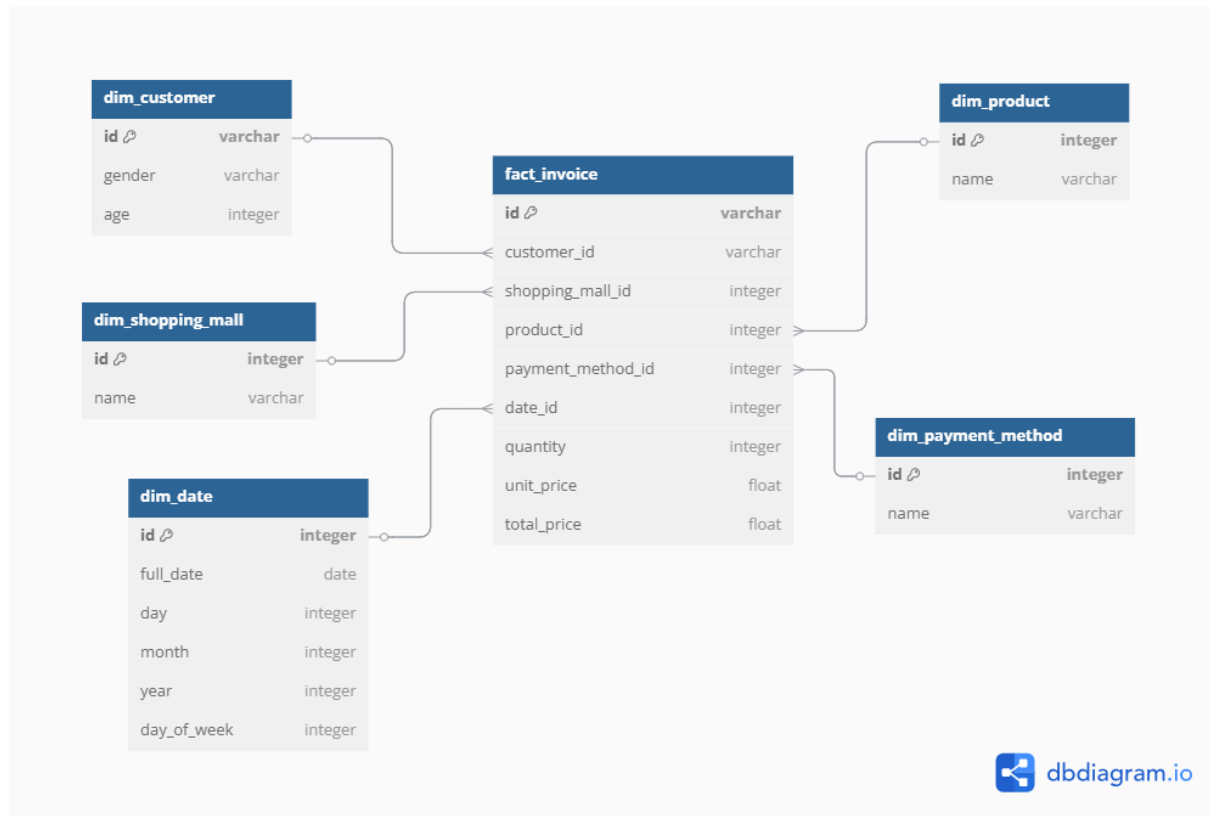
Dependencias utilizadas

1. pandas: para extraer y transformar los datos del csv
2. sqlalchemy: para crear las tablas y cargar los datos a la base de datos
3. python-dotenv: para las variables de entorno
4. psycopg2-binary: para la conexión de la base de datos con PostgreSQL
5. matplotlib: para generar las gráficas de las consultas

II. Diseño del modelo de bodega de datos

El modelo de bodega de base de datos seleccionado es **estrella**, porque la cantidad de datos cuantitativos, es limitada y no requiere de mayor cantidad de tablas para organizar y/o almacenar la información proporcionada.

Diagrama de tablas (Modelo entidad relación de la DB)



Ventajas del modelo estrella

1. Los modelos estrella son fáciles de entender e implementar
2. El rendimiento es mucho más rápido ya que permite rápidas agregaciones y un acceso optimizado a los datos
3. Los modelos estrella permiten un análisis de datos más intuitivo, facilitando el uso por analistas y herramientas BI como Power BI o Tableau
4. Permite la escalabilidad, se pueden agregar nuevas dimensiones sin afectar el rendimiento

Análisis de las tablas

Tabla dim_customer (tabla de dimensión de clientes)

Campo	Tipo	Descripción
id	varchar	Identificador único del cliente
gender	varchar	Género del cliente
age	integer	Edad del cliente

Tabla dim_product (tabla de dimensión de productos)

Campo	Tipo	Descripción
id	integer	Identificador único del producto.
name	varchar	Nombre de la categoría

Tabla dim_payment_method (tabla de dimensión de métodos de pago)

Campo	Tipo	Descripción
id	integer	Identificador único del método de pago
name	varchar	Nombre del método

Tabla dim_shopping_mall (tabla de dimensión de centros comerciales)

Campo	Tipo	Descripción
id	integer	Identificador único del centro comercial
name	varchar	Nombre del centro comercial

Tabla dim_date (tabla de dimensión de fechas de las facturas)

Campo	Tipo	Descripción
id	integer	Identificador único de la fecha
full_date	date	Fecha completa
day	integer	Día del mes
month	integer	Mes del año
year	integer	Año
day_of_week	integer	Día de la semana (ej. 1=Lunes)

Tabla fact_invoice (tabla de hechos de facturas)

Campo	Tipo	Descripción
id	varchar	Identificador único de la factura
customer_id	varchar	Cliente que realizó la compra (FK a dim_customer)
shopping_mall_id	integer	Centro comercial donde ocurrió la compra (FK a dim_shopping_mall)
product_id	integer	Producto comprado (FK a dim_product)
payment_method_id	integer	Método de pago usado (FK a dim_payment_method)
date_id	integer	Fecha de la transacción (FK a dim_date)
quantity	integer	Cantidad de productos comprados
unit_price	float	Precio por unidad del producto
total_price	float	Precio total de la compra

III. Extracción, transformación y carga de datos

Proceso de transformación y carga de datos a la base de datos en PostgreSQL

Se analizaron los datos usando pandas del archivo csv, para verificar el nombre de las columnas, si hay datos nulos y los tipos de datos para cada columna

```
# ver la informacion de los datos
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 99457 entries, 0 to 99456
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   invoice_no            99457 non-null  object
1   customer_id           99457 non-null  object
2   gender                99457 non-null  object
3   age                   99457 non-null  int64
4   category              99457 non-null  object
5   quantity              99457 non-null  int64
6   price                 99457 non-null  float64
7   payment_method        99457 non-null  object
8   invoice_date          99457 non-null  object
9   shopping_mall         99457 non-null  object
dtypes: float64(1), int64(2), object(7)
memory usage: 7.6+ MB
```

Se puede ver con csv contiene 10 columnas las cuales son:

1. invoice_no: el número de la factura
2. customer_id: el id del cliente
3. gender: el género del cliente
4. age: la edad del cliente
5. category: la categoría del producto
6. quantity: la cantidad llevada del cliente
7. price: el precio del producto
8. payment_method: método de pago utilizado
9. invoice_date: fecha de la factura
10. shopping_mall: centro comercial o tienda

Hay 1 columna con el tipo de dato float, hay 2 columnas con el tipo de dato int y hay 7 columnas con el tipo de datos object string

Las columnas no tienen valores nulos

```
# verificar si hay valores nulos  
df.isnull().sum()
```

```
invoice_no      0  
customer_id     0  
gender          0  
age             0  
category        0  
quantity        0  
price           0  
payment_method  0  
invoice_date    0  
shopping_mall   0  
dtype: int64
```

No hay registros duplicados

```
# verificar si hay valores duplicados  
df.duplicated().any()
```

```
np.False_
```

Se puede ver que los tipos de datos corresponden a la columna excepto para la columna `invoice_date` que tiene el tipo de dato `object` string, por lo tanto se debe hacer una transformación para el tipo de datos fecha

```
# transformar la columna invoice_date a fecha
df["invoice_date"] = pd.to_datetime(df["invoice_date"], dayfirst=True)
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 99457 entries, 0 to 99456
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   invoice_no            99457 non-null  object
1   customer_id           99457 non-null  object
2   gender                99457 non-null  object
3   age                   99457 non-null  int64
4   category              99457 non-null  object
5   quantity              99457 non-null  int64
6   price                 99457 non-null  float64
7   payment_method        99457 non-null  object
8   invoice_date          99457 non-null  datetime64[ns]
9   shopping_mall         99457 non-null  object
dtypes: datetime64[ns](1), float64(1), int64(2), object(6)
memory usage: 7.6+ MB
```

Con la transformación, hay 1 columna con el tipo de dato datetime, hay 1 columna con el tipo de dato int, hay 2 columnas con el tipo de dato int y hay 6 columnas con el tipo de datos object string

Crear las tablas de la base de datos

Configuración de las variables de entorno para la conexión con la base de datos


```

import os
from dotenv import load_dotenv

load_dotenv()

# variables de entorno
class GlobalConfig:
    def __init__(self):
        self.DB_USER = os.environ.get("DB_USER")
        self.DB_PASSWORD = os.environ.get("DB_PASSWORD")
        self.DB_HOST = os.environ.get("DB_HOST", "localhost")
        self.DB_PORT = int(os.environ.get("DB_PORT", 5432))
        self.DB_NAME = os.environ.get("DB_NAME")

global_config = GlobalConfig()

```

Conexión a la base de datos con PostgreSQL

```

from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base
from config.global_config import global_config

# URL de la base de datos
DATABASE_URL = f"postgresql://{global_config.DB_USER}:{global_config.DB_PASSWORD}@{global_config.DB_HOST}:{global_config.DB_PORT}/{global_config.DB_NAME}"

# motor de base de datos
engine = create_engine(DATABASE_URL, echo=True)

# sesión de base de datos
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

# base de datos para modelos
Base = declarative_base()

```

Modelo para los productos

```

from sqlalchemy import Column, Integer, String
from sqlalchemy.orm import relationship
from db.database import Base

class DimProduct(Base):
    __tablename__ = "dim_product"

    id = Column(Integer, primary_key=True)
    name = Column(String) # nombre de la categoría

    invoices = relationship("FactInvoice", back_populates="product", cascade="all, delete")

```

Modelo para los clientes

```

from sqlalchemy import Column, Integer, String
from sqlalchemy.orm import relationship
from db.database import Base

class DimCustomer(Base):
    __tablename__ = "dim_customer"

    id = Column(String, primary_key=True)
    gender = Column(String) # género
    age = Column(Integer) # edad

    invoices = relationship("FactInvoice", back_populates="customer", cascade="all, delete")

```

Modelo para las fechas de las facturas

```

from sqlalchemy import Column, Integer, Date
from sqlalchemy.orm import relationship
from db.database import Base

class DimDate(Base):
    __tablename__ = "dim_date"

    id = Column(Integer, primary_key=True)
    full_date = Column(Date) # fecha completa
    day = Column(Integer) # día
    month = Column(Integer) # mes
    year = Column(Integer) # año
    day_of_week = Column(Integer) # día de la semana

    invoices = relationship("FactInvoice", back_populates="date", cascade="all, delete")

```

Modelo para los métodos de pagos de las facturas

```

from sqlalchemy import Column, Integer, String
from sqlalchemy.orm import relationship
from db.database import Base

class DimPaymentMethod(Base):
    __tablename__ = "dim_payment_method"

    id = Column(Integer, primary_key=True)
    name = Column(String) # nombre del método de pago

    invoices = relationship("FactInvoice", back_populates="payment_method", cascade="all, delete")

```

Modelo para los centros comerciales o tiendas

```

from sqlalchemy import Column, Integer, String
from sqlalchemy.orm import relationship
from db.database import Base

class DimShoppingMall(Base):
    __tablename__ = "dim_shopping_mall"

    id = Column(Integer, primary_key=True)
    name = Column(String) # nombre del centro comercial

    invoices = relationship("FactInvoice", back_populates="shopping_mall", cascade="all, delete")

```

Modelo para las facturas

```

from sqlalchemy import Column, Integer, String, Float, ForeignKey
from sqlalchemy.orm import relationship
from db.database import Base

class FactInvoice(Base):
    __tablename__ = "fact_invoice"

    id = Column(String, primary_key=True)
    quantity = Column(Integer) # cantidad del producto
    unit_price = Column(Float) # precio unitario del producto
    total_price = Column(Float) # precio total de la compra
    customer_id = Column(String, ForeignKey("dim_customer.id")) # id del cliente
    product_id = Column(Integer, ForeignKey("dim_product.id")) # id del producto
    payment_method_id = Column(Integer, ForeignKey("dim_payment_method.id")) # id del método de pago
    shopping_mall_id = Column(Integer, ForeignKey("dim_shopping_mall.id")) # id del centro comercial
    date_id = Column(Integer, ForeignKey("dim_date.id")) # id de la fecha

    customer = relationship("DimCustomer", back_populates="invoices")
    product = relationship("DimProduct", back_populates="invoices")
    payment_method = relationship("DimPaymentMethod", back_populates="invoices")
    shopping_mall = relationship("DimShoppingMall", back_populates="invoices")
    date = relationship("DimDate", back_populates="invoices")

```

Crear las tablas de forma automática en la base de datos

```
# crear las tablas en la base de datos
try:
    Base.metadata.create_all(bind=engine)
    print("Tablas creadas correctamente")
except Exception as e:
    print("Error al crear las tablas:", e)
```

Proceso para cargar los datos a la base de datos

Cargar los datos del csv

```
# cargar datos del dataset
def load_data(file_path):
    return pd.read_csv(file_path)
```

Guardar los datos de los productos, haciendo una consulta al dataframe donde obtiene el nombre de las categorías

```
# cargar datos de productos
def insert_products(db, dataframe):
    if db.query(DimProduct).count() == 0:
        # obtener productos únicos
        products = [{"name": product} for product in dataframe["category"].unique()]
        db.bulk_insert_mappings(DimProduct, products)
```

Guardar los datos de los métodos de pagos de las facturas, haciendo una consulta al dataframe donde obtiene el nombre de los métodos de pagos sin repetir

```
# cargar datos de metodos de pago
def insert_payment_methods(db, dataframe):
    if db.query(DimPaymentMethod).count() == 0:
        payment_methods = [{"name": method} for method in dataframe["payment_method"].unique()]
        db.bulk_insert_mappings(DimPaymentMethod, payment_methods)
```

Guarda los datos de los centros comerciales de cada factura, haciendo una consulta al dataframe donde se obtiene el nombre de los centros comerciales sin repetir

```
# cargar datos de centros comerciales
def insert_shopping_malls(db, dataframe):
    if db.query(DimShoppingMall).count() == 0:
        shopping_malls = [{"name": mall} for mall in dataframe["shopping_mall"].unique()]
        db.bulk_insert_mappings(DimShoppingMall, shopping_malls)
```

Guardar los datos de fechas de las facturas, donde se transforma los valores de la columna invoice_date de object a datetime y de esta forma extraer los valores del día, mes, año y día de la semana, por último se extraen del dataframe las fechas sin repetir

```
# cargar datos de fechas
def insert_dates(db, dataframe):
    if db.query(DimDate).count() == 0:
        # transformar la columna invoice_date a tipo fecha
        dataframe['invoice_date'] = pd.to_datetime(dataframe['invoice_date'], dayfirst=True)
        dates = [
            {
                "full_date": date,
                "day": date.day,
                "month": date.month,
                "year": date.year,
                "day_of_week": date.dayofweek
            }
            for date in dataframe["invoice_date"].unique()
        ]
        db.bulk_insert_mappings(DimDate, dates)
```

Guardar los datos de los clientes de cada factura, se extraen los datos del cliente de dataframe borrando los valores duplicados, en este caso se extrae el id del cliente, el género y la edad

```
# cargar datos de clientes
def insert_customers(db, dataframe):
    if db.query(DimCustomer).count() == 0:
        customers = dataframe[["customer_id", "gender", "age"]].drop_duplicates()
        customers_data = [
            {
                "id": row["customer_id"],
                "gender": row["gender"],
                "age": row["age"]
            }
            for _, row in customers.iterrows()
        ]
        db.bulk_insert_mappings(DimCustomer, customers_data)
```

Guardar los datos de cada factura, se mapean los id (claves foráneas) de cada tabla en diccionarios para optimizar el código, luego se itera el dataframe para guarda cada factura y usando los diccionarios se obtiene los id de cada tabla

```
# cargar datos de facturas
def insert_invoices(db, dataframe):
    if db.query(FactInvoice).count() == 0:
        # mapear los IDs de las tablas relacionadas
        product_map = {p.name: p.id for p in db.query(DimProduct).all()}
        payment_map = {p.name: p.id for p in db.query(DimPaymentMethod).all()}
        mall_map = {m.name: m.id for m in db.query(DimShoppingMall).all()}
        customer_map = {c.id: c.id for c in db.query(DimCustomer).all()}
        date_map = {d.full_date: d.id for d in db.query(DimDate).all()}

        # crear una nueva columna con el total de la factura
        dataframe["total_price"] = dataframe["price"] * dataframe["quantity"]

        # transformar la columna invoice_date a tipo fecha
        dataframe['invoice_date'] = pd.to_datetime(dataframe['invoice_date'], dayfirst=True)

        invoices_data = [
            {
                "id": row["invoice_no"],
                "customer_id": customer_map[row["customer_id"]],
                "product_id": product_map[row["category"]],
                "payment_method_id": payment_map[row["payment_method"]],
                "shopping_mall_id": mall_map[row["shopping_mall"]],
                "date_id": date_map[row["invoice_date"].date()],
                "quantity": row["quantity"],
                "unit_price": row["price"],
                "total_price": row["total_price"]
            }
            for _, row in dataframe.iterrows()
        ]
        db.bulk_insert_mappings(FactInvoice, invoices_data)
```

Guardar los datos a la base de datos

```

# cargar datos en la base de datos
def load_db(db):
    dataFrame = load_data(csv_file)

    try:
        insert_products(db, dataFrame)
        insert_payment_methods(db, dataFrame)
        insert_shopping_malls(db, dataFrame)
        insert_customers(db, dataFrame)
        insert_dates(db, dataFrame)
        insert_invoices(db, dataFrame)

        # confirmar cambios
        db.commit()
        print("Datos insertados correctamente")
    except Exception as e:
        db.rollback()
        print(f"Error: {e}")

    finally:
        db.close()
        print("Conexión cerrada")

```

Comprobación de los resultados en pgAdmin con PostgreSQL:

Tablas creadas en la base de datos

- ▼  Tables (6)
 - >  dim_customer
 - >  dim_date
 - >  dim_payment_method
 - >  dim_product
 - >  dim_shopping_mall
 - >  fact_invoice

Datos de los productos

Query Query History

```
1 SELECT * FROM public.dim_product
2 ORDER BY id ASC
```

Data Output Messages Notifications

	id [PK] integer	name character varying
1	1	Clothing
2	2	Shoes
3	3	Books
4	4	Cosmetics
5	5	Food & Beverage
6	6	Toys
7	7	Technology
8	8	Souvenir

Datos de los clientes

Query

Query History

1

2

SELECT * FROM public.dim_customer

ORDER BY id ASC

Data Output

Messages

Notifications

</

Datos de las fecha de las facturas

Query

Query History

1

2

SELECT * FROM public.dim_date

ORDER BY id ASC

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

	id [PK] integer	full_date date	day integer	month integer	year integer	day_of_week integer
1	1	2022-08-05	5	8	2022	4
2	2	2021-12-12	12	12	2021	6
3	3	2021-11-09	9	11	2021	1
4	4	2021-05-16	16	5	2021	6
5	5	2021-10-24	24	10	2021	6
6	6	2022-05-24	24	5	2022	1
7	7	2022-03-13	13	3	2022	6
8	8	2021-01-13	13	1	2021	2
9	9	2021-11-04	4	11	2021	3
10	10	2021-08-22	22	8	2021	6

Datos de los métodos de pago

Query

Query History

1

2

SELECT * FROM public.dim_payment_method
ORDER BY id ASC

Data Output

Messages

Notifications

≡+

▼

▼

	id [PK] integer	name character varying
1	1	Credit Card
2	2	Debit Card
3	3	Cash

Datos de los centros comerciales

Query		Query History		
1	SELECT * FROM public.dim_shopping_mall			
2				
Data Output		Messages	Notifications	
				
				
				
	id [PK] integer 	name character varying 		
1	1	Kanyon		
2	2	Forum Istanbul		
3	3	Metrocity		
4	4	Metropol AVM		
5	5	Istinye Park		
6	6	Mall of Istanbul		
7	7	Emaar Square Mall		
8	8	Cevahir AVM		
9	9	Viaport Outlet		
10	10	Zorlu Center		

Datos de las facturas con sus respectivas llaves foráneas

Query Query History

```
1 SELECT c.name as category, SUM(i.total_price) as total
2 FROM fact_invoice i
3 JOIN dim_product c ON i.product_id = c.id
4 GROUP BY c.name
5 ORDER BY total DESC;
```

Data Output Messages Notifications

	category character varying 	total double precision 
1	Clothing	113996791.03997
2	Shoes	66553451.47000195
3	Technology	57862350
4	Cosmetics	6792862.8999999715
5	Toys	3980426.239999814
6	Food & Beverage	849535.0500000005
7	Books	834552.8999999997
8	Souvenir	635824.6499999914

Consulta a la base de datos para obtener los métodos de pagos más utilizados ordenados de mayor a menor

Query Query History

```
1 SELECT p.name as payment_method, COUNT(i.id) as total
2 FROM fact_invoice i
3 JOIN dim_payment_method p ON i.payment_method_id = p.id
4 GROUP BY p.name
5 ORDER BY total DESC;
```

Data Output Messages Notifications

	payment_method character varying	total bigint
1	Cash	44447
2	Credit Card	34931
3	Debit Card	20079

Consulta para obtener el total de ventas por método de pago ordenados de mayor a menor

Query Query History

```
1 SELECT p.name as payment_method, SUM(i.total_price) as total
2 FROM fact_invoice i
3 JOIN dim_payment_method p ON i.payment_method_id = p.id
4 GROUP BY p.name
5 ORDER BY total DESC;
```

Data Output Messages Notifications

	payment_method character varying	total double precision
1	Cash	112832243.01999287
2	Credit Card	88077123.76999618
3	Debit Card	50596427.459999315

Consulta para obtener los centros comerciales más visitados ordenados de mayor a menor

Query Query History

```
1 SELECT s.name as shopping_mall, COUNT(i.id) as total
2 FROM fact_invoice i
3 JOIN dim_shopping_mall s ON i.shopping_mall_id = s.id
4 GROUP BY s.name
5 ORDER BY total DESC;
```

Data Output Messages Notifications

	shopping_mall character varying 	total bigint 
1	Mall of Istanbul	19943
2	Kanyon	19823
3	Metrocity	15011
4	Metropol AVM	10161
5	Istinye Park	9781
6	Zorlu Center	5075
7	Cevahir AVM	4991
8	Forum Istanbul	4947
9	Viaport Outlet	4914
10	Emaar Square Mall	4811

Consulta para obtener el total de ventas por centro comercial ordenados de mayor a menor

Query

Query History

1

SELECT s.name as shopping_mall, SUM(i.total_price) as total

2

FROM fact_invoice i

3

JOIN dim_shopping_mall s ON i.shopping_mall_id = s.id

4

GROUP BY s.name

5

ORDER BY total DESC;

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

	shopping_mall character varying 🔒	total double precision 🔒
1	Mall of Istanbul	50872481.67999938
2	Kanyon	50554231.09999922
3	Metrocity	37302787.33000004
4	Metropol AVM	25379913.190000214
5	Istinye Park	24618827.680000238
6	Zorlu Center	12901053.820000079
7	Cevahir AVM	12645138.200000118
8	Viaport Outlet	12521339.720000127
9	Emaar Square Mall	12406100.290000135
10	Forum Istanbul	12303921.240000157

Consulta para obtener los clientes que más compran ordenados por el total de mayor a menor

Query

Query History

1

2

3

4

5

6

SELECT c.id as customer, SUM(i.total_price) as total

FROM fact_invoice i

JOIN dim_customer c ON i.customer_id = c.id

GROUP BY c.id

ORDER BY total DESC;

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

	customer character varying 🔒	total double precision 🔒
1	C326699	26250
2	C287416	26250
3	C330951	26250
4	C295182	26250
5	C239972	26250
6	C313133	26250
7	C571399	26250
8	C260187	26250
9	C796010	26250
10	C157996	26250

Consulta para obtener el total de ventas por mes ordenado por el total de ventas de mayor a menor

Query Query History

```
1 SELECT d.month as month, SUM(i.total_price) as total
2 FROM fact_invoice i
3 JOIN dim_date d ON i.date_id = d.id
4 GROUP BY d.month
5 ORDER BY total DESC;
6
```

Data Output Messages Notifications

	month integer		total double precision	
1	1		28891525.590000425	
2	2		26625090.100000367	
3	3		21956191.330000026	
4	10		20545090.430000182	
5	7		20378722.630000226	
6	5		19719331.10000023	
7	12		19455085.640000213	
8	8		19282361.290000148	
9	6		18933775.300000028	
10	9		18795794.910000216	
11	4		18715685.980000021	
12	11		18207139.950000245	

Consulta para obtener el total de ventas por el dia de la semana ordenados de mayor a menor

Query

Query History

1

SELECT d.day_of_week as day_of_week, SUM(i.total_price) as total

2

FROM fact_invoice i

3

JOIN dim_date d ON i.date_id = d.id

4

GROUP BY d.day_of_week

5

ORDER BY total DESC;

6

Data Output

Messages

Notifications

≡+

▼

▼

	day_of_week integer	total double precision
1	0	37296648.11000023
2	1	36298096.66000042
3	3	35738148.630000204
4	4	35728331.39000023
5	6	35689090.61000005
6	2	35575650.13000009
7	5	35179828.72000016

Consulta para obtener el total de ventas por género del cliente

Query Query History

```
1 SELECT c.gender as gender, SUM(i.total_price) as total
2 FROM fact_invoice i
3 JOIN dim_customer c ON i.customer_id = c.id
4 GROUP BY c.gender
5 ORDER BY total DESC;
6
```

Data Output Messages Notifications

	gender character varying 	total double precision 
1	Female	150207136.01998436
2	Male	101298658.22999445

Consulta para obtener el género del cliente que más compra

Query Query History

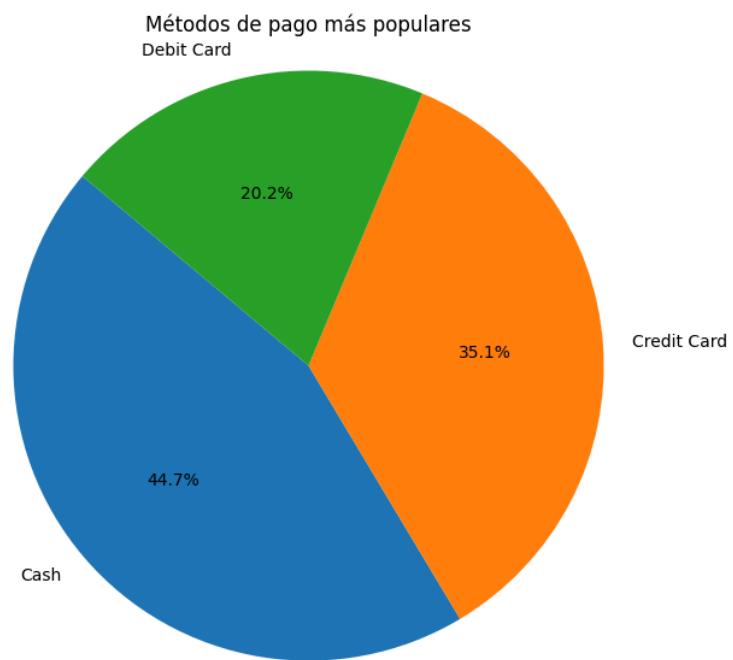
```
1 SELECT c.gender as gender, COUNT(i.total_price) as total
2 FROM fact_invoice i
3 JOIN dim_customer c ON i.customer_id = c.id
4 GROUP BY c.gender
5 ORDER BY total DESC;
6
```

Data Output Messages Notifications

	gender character varying 	total bigint 
1	Female	59482
2	Male	39975

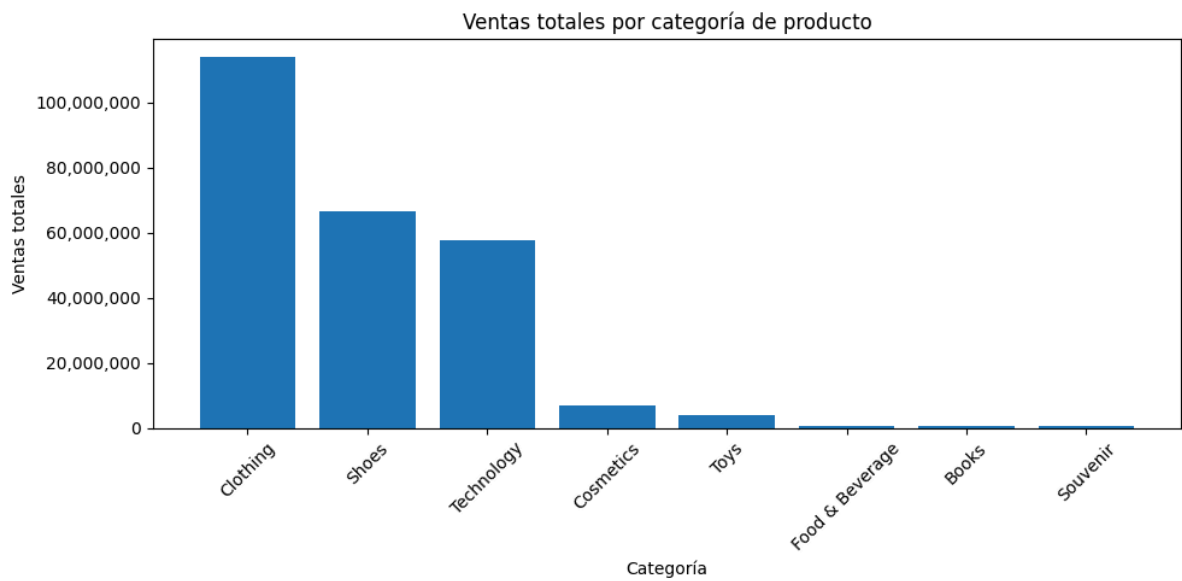
V. Análisis descriptivo y visualización de datos

Gráfica de los métodos de pagos más utilizados



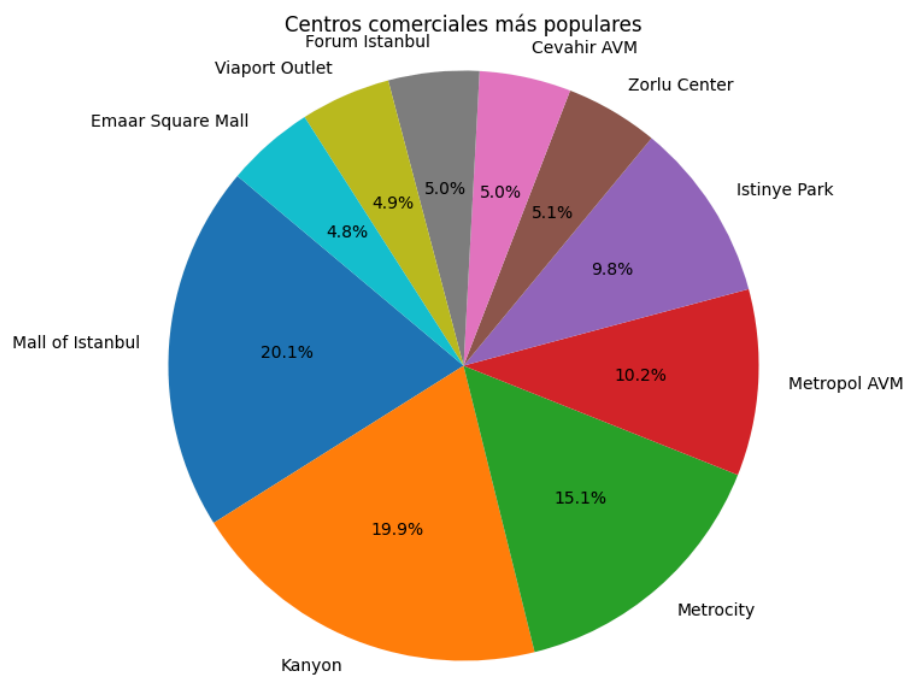
El método de pago más utilizado es Cash, seguido de Credit Card y por último Debit Card

Gráfica del total de ventas por categoría de productos

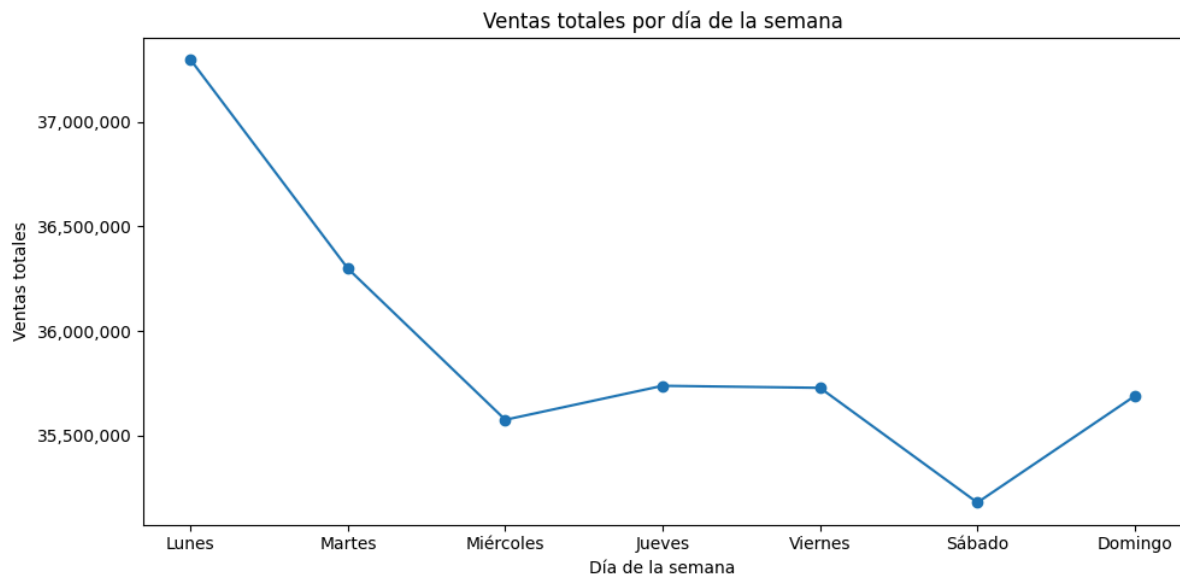


Las tres categorías más vendidas son Clothing, seguido de Shoes y Technology

Gráfica de los centro comerciales más utilizados

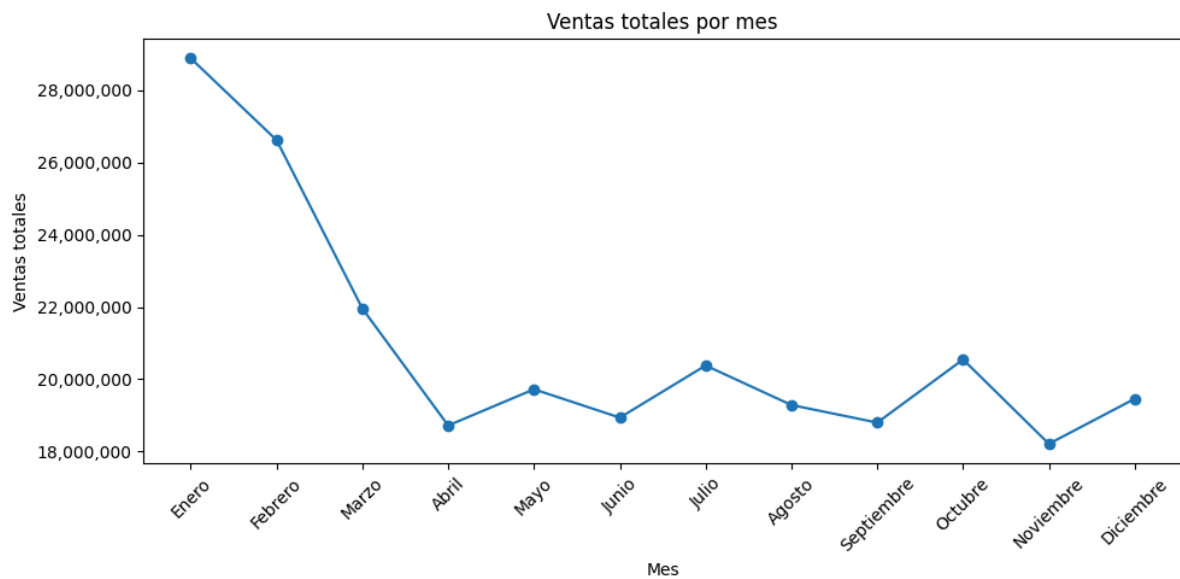


Los dos centros comerciales más frecuentados son Mall of Istanbul y Kanyon
Gráfica del total de venta por día de la semana



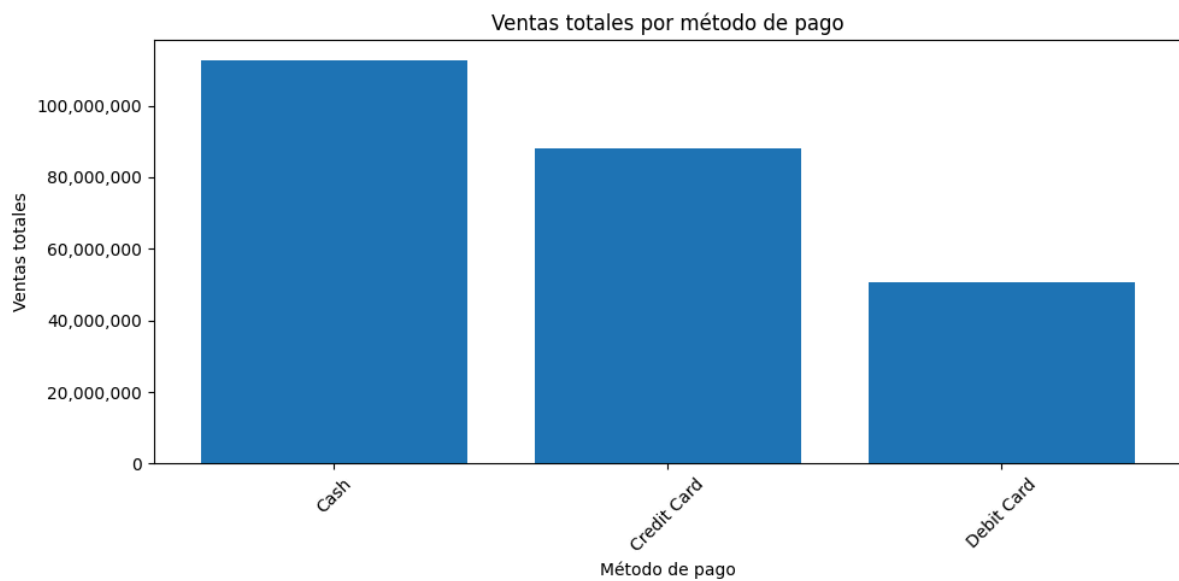
Los dos días de la semana donde más se compra son los días Lunes y Martes

Gráfica del total de ventas por mes



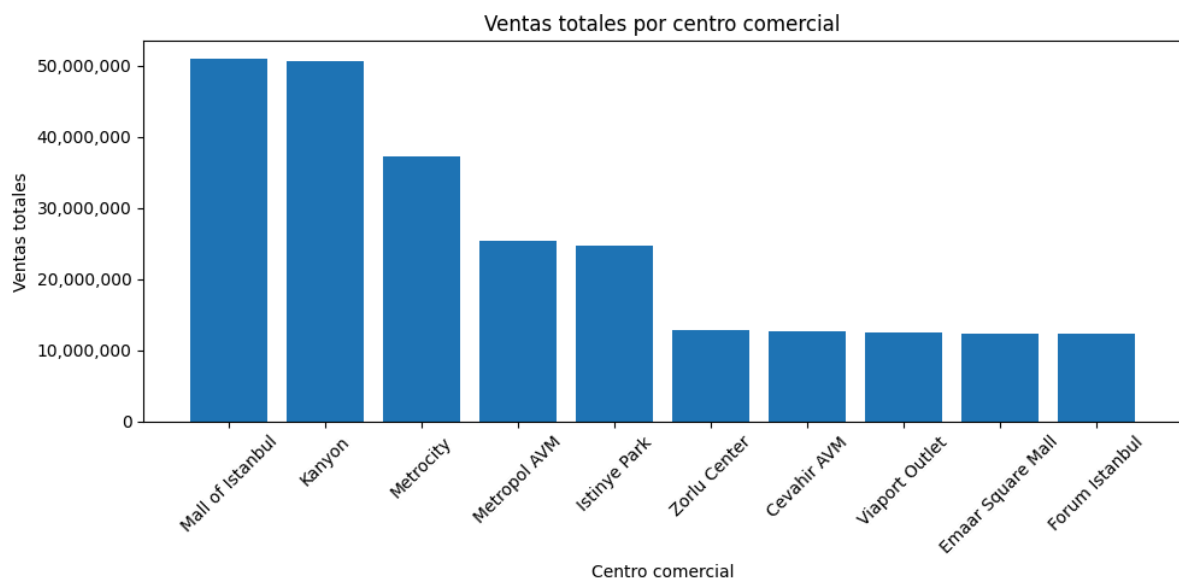
Los tres meses con mayores ganancias son los meses de Enero, seguido de Febrero y Marzo

Gráfica del total de ventas por método de pago



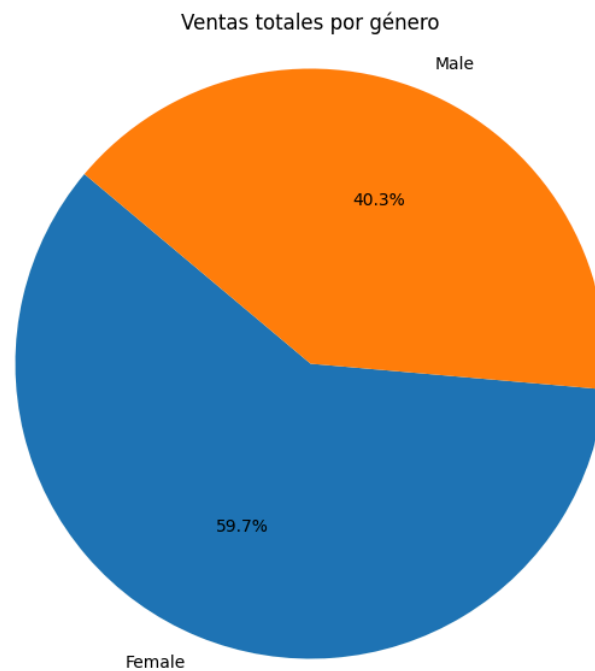
El método de pago con mayores ventas es Cash

Gráfica del total de ventas por centro comercial



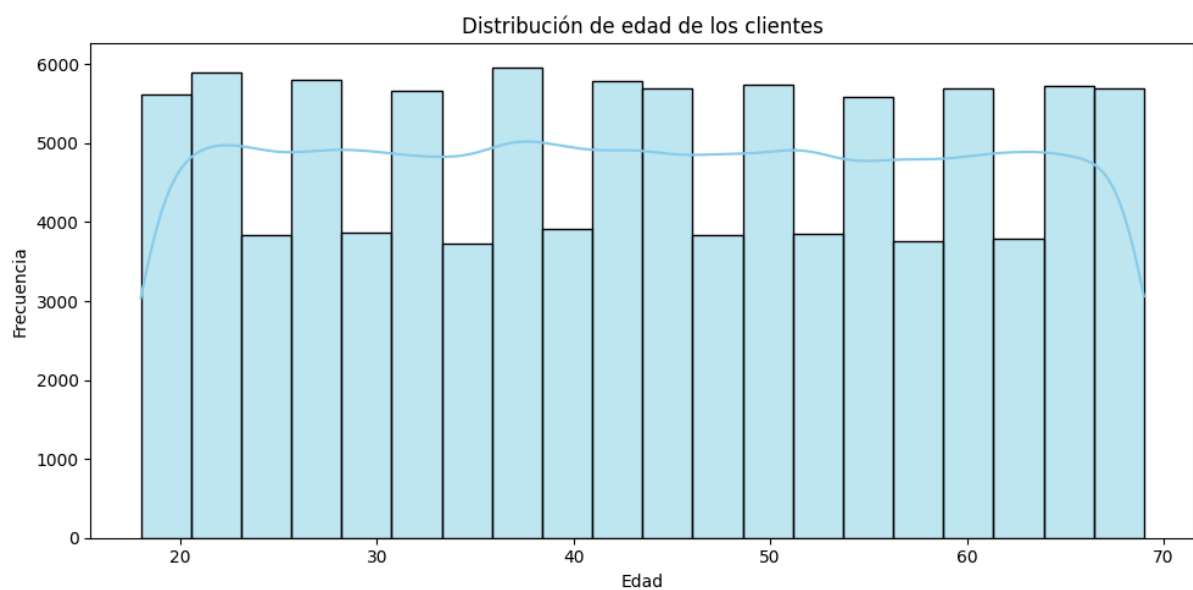
Los tres centro comerciales que más ventas generan son Mall of Istanbul, seguido de Kanyon y Metrocity

Gráfica del total de ventas por género del cliente



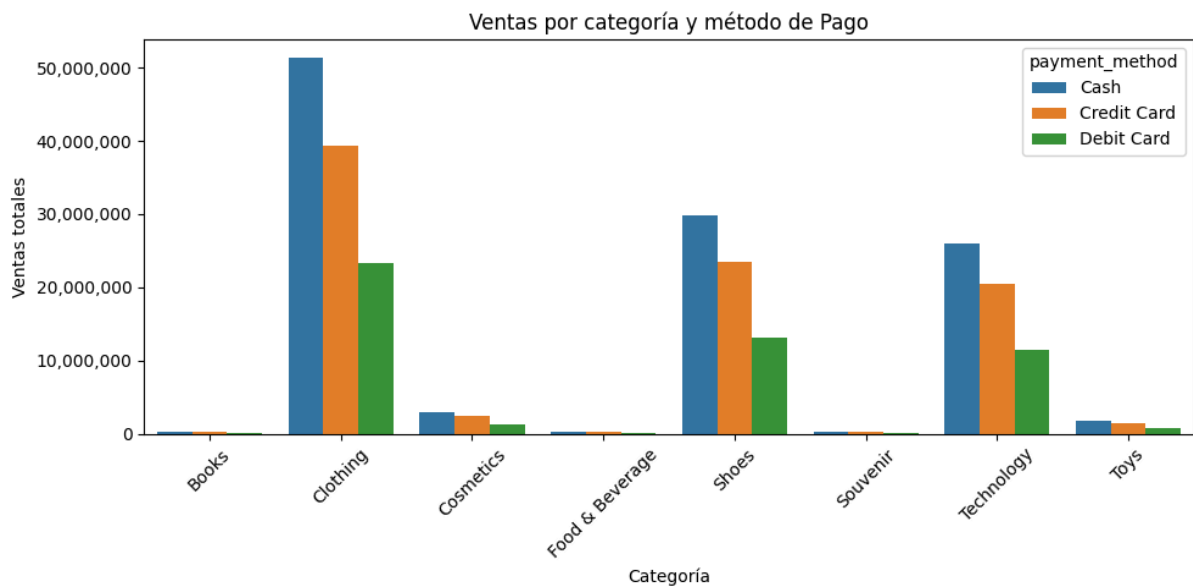
Las mujeres son las que más compran

Gráfica distribución de edad de los clientes



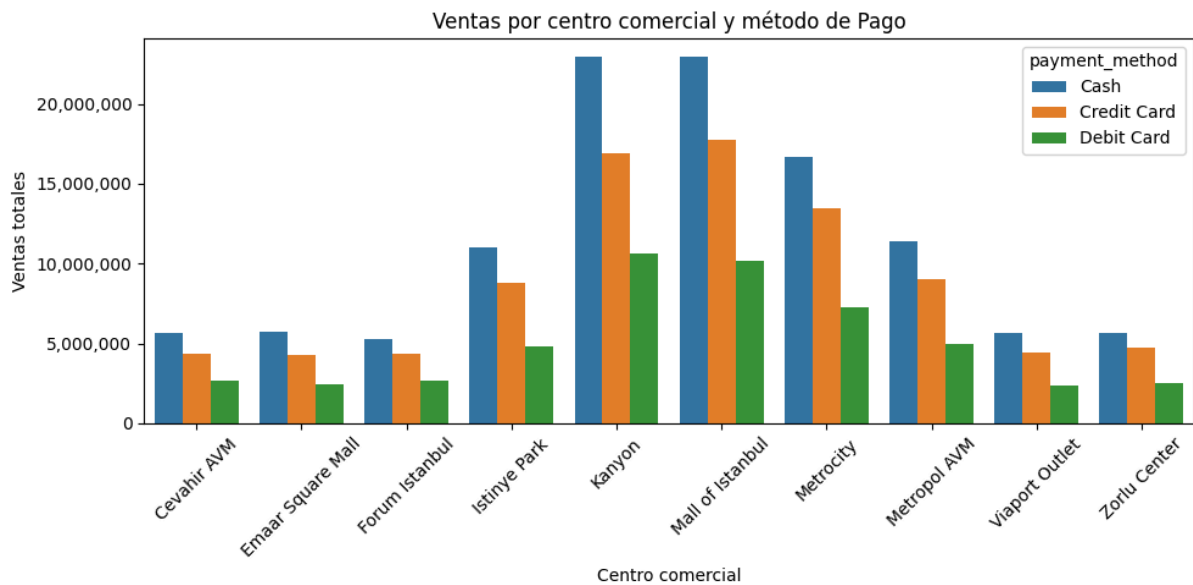
La frecuencia es relativamente uniforme en la mayoría de los grupos de edad, con una ligera concentración en los rangos de 20 a 30 años y de 50 a 60 años.

Gráfica de ventas por categoría y métodos de pago



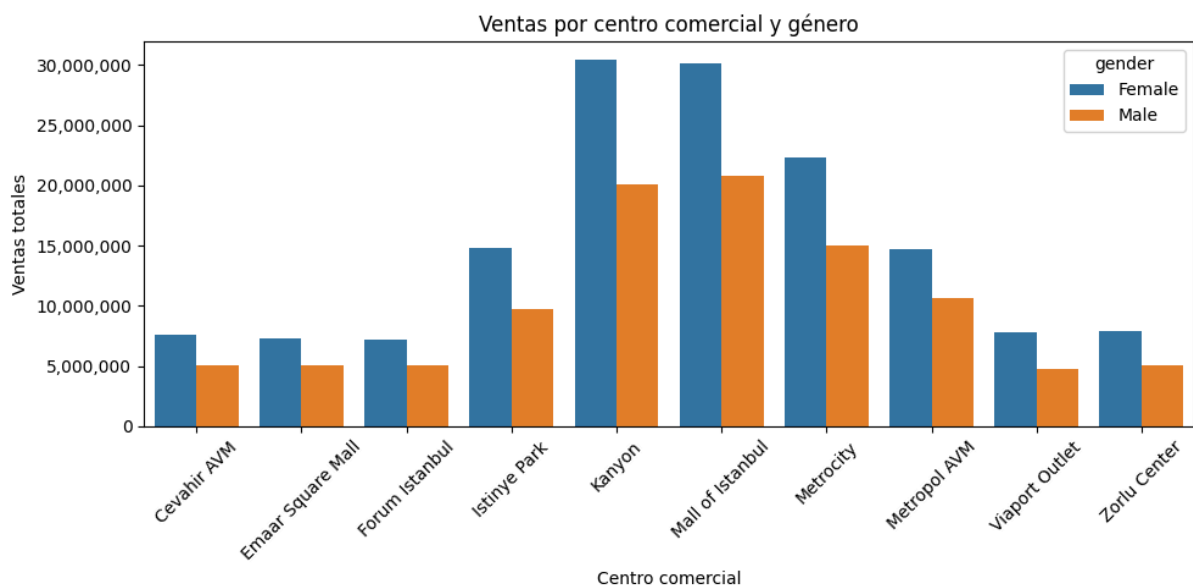
Se observa que la categoría Clothing es la más vendida, con Cash como principal método de pago

Gráfica de ventas por centro comercial y métodos de pago



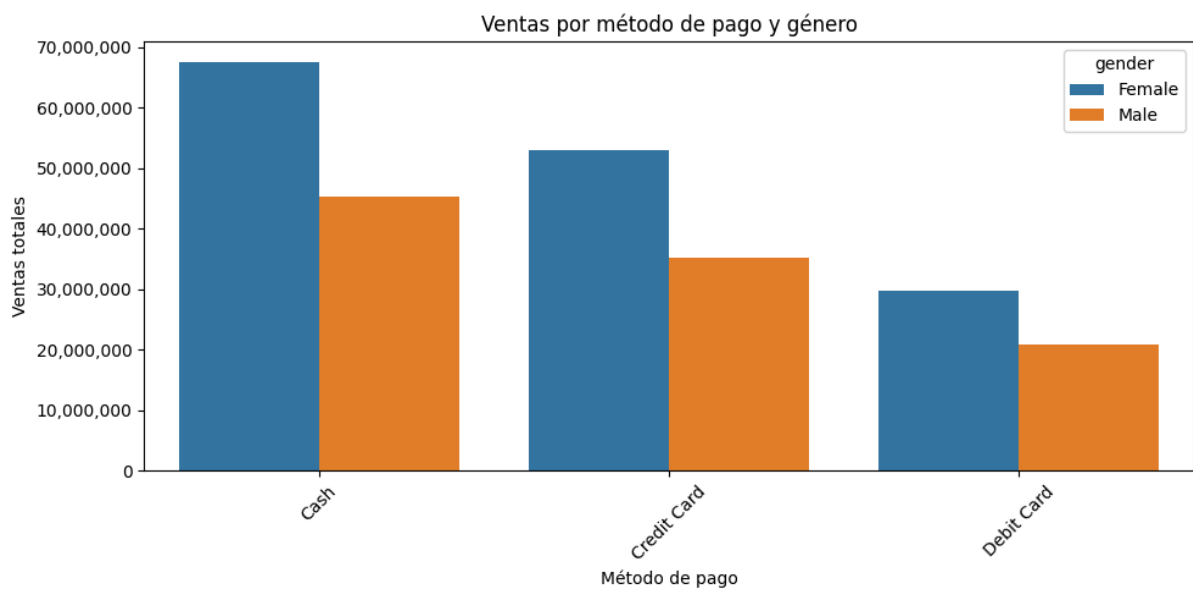
Se observa que los centros comerciales Mall of Istanbul y Kanyon presentan datos con similitud, son los dos centros comerciales que mayores ganancias tienen y el método de pago más usado es Cash

Gráfica de ventas por centro comercial y género



Se observa que las mujeres son las que más compran en todos los centros comerciales siendo Mall of Istanbul y Kanyon donde más compran

Gráfica de ventas por métodos de pagos y género



El método de pago más utilizado por las mujeres y hombres es el Cash

VI. Conclusiones

En este proyecto, se aplicaron varios conceptos clave de ciencia de datos, desde la creación de una bodega de datos hasta la visualización de resultados.

- **Modelo de Datos:** Se diseñó una bodega de datos en PostgreSQL usando el modelo estrella, lo que ayudó a organizar la información de forma eficiente para hacer consultas.
- **ETL:** Se usó Pandas y SQLAlchemy para extraer, transformar y cargar los datos en la base de datos, automatizando el proceso de integración de datos.
- **Consultas y Análisis:** Se crearon consultas en SQL para explorar los datos y aplicar análisis descriptivo que permitió entender mejor las tendencias.
- **Visualización:** Se crearon gráficos con Matplotlib para representar los datos de forma más clara y facilitar su interpretación.